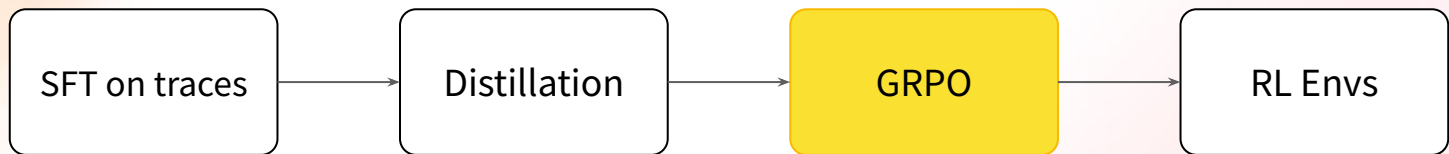


TRAINING AGENTS

JULY 28th

class 3:
reinforcement learning

Where we are in the series



Class 1 and 2 taught the model to **imitate**.

- Class 1: imitate the demonstrator's traces, token by token
- Class 2: Imitate the teacher's distribution

Both are imitation, and it has a ceiling, the demonstrator



What changes today: the signal

SFT → a target token at every position (dense)

Distillation → the teacher's distribution at every position (denser)

RL → one number per attempt: the reward (sparse)

The chess player from Class 2: stop studying the coach's games and play your own. Nobody shows you the move, you get one signal per game (win/lose). That is enough to improve beyond anyone you could imitate.

The signal tells the model how well it did



How GRPO works



We will cover

The loop

Policy, rollout, reward, update: and what one GRPO round actually does

The group baseline

Where the learning signal comes from, and why there is no value model

Two guardrails

Clipping bounds one reused batch; the KL penalty bounds the whole run

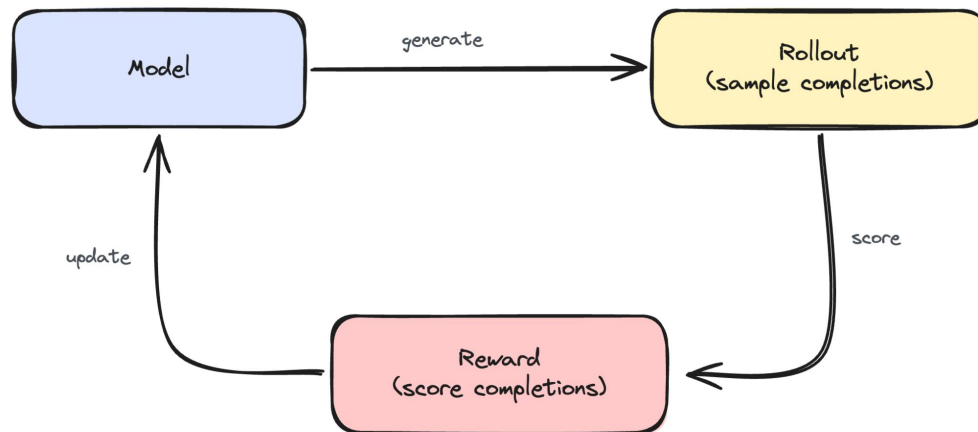
The reward contract

What a verifier can see, and why it must produce variation

Running it

The same loop in TRL, and how to read a run that is going wrong

Reinforcement learning, in three functions



Policy

The model. Maps a prompt to a distribution over completions. This is what you train.

Rollout

Sampling completions from the policy. The model's own attempts at the task.

Reward

A function from a completion to a number. Higher is better.

RL: play, no coach

you play your own games



one signal per match: won or lost -
nothing about which moves mattered

on-policy - sparse

Off-policy KD: study the master

grandmaster's games,
annotated



every move explained - but these are
positions you will rarely reach yourself

off-policy - dense

OPD: coach grades your games

you play, a coach reviews
every single move

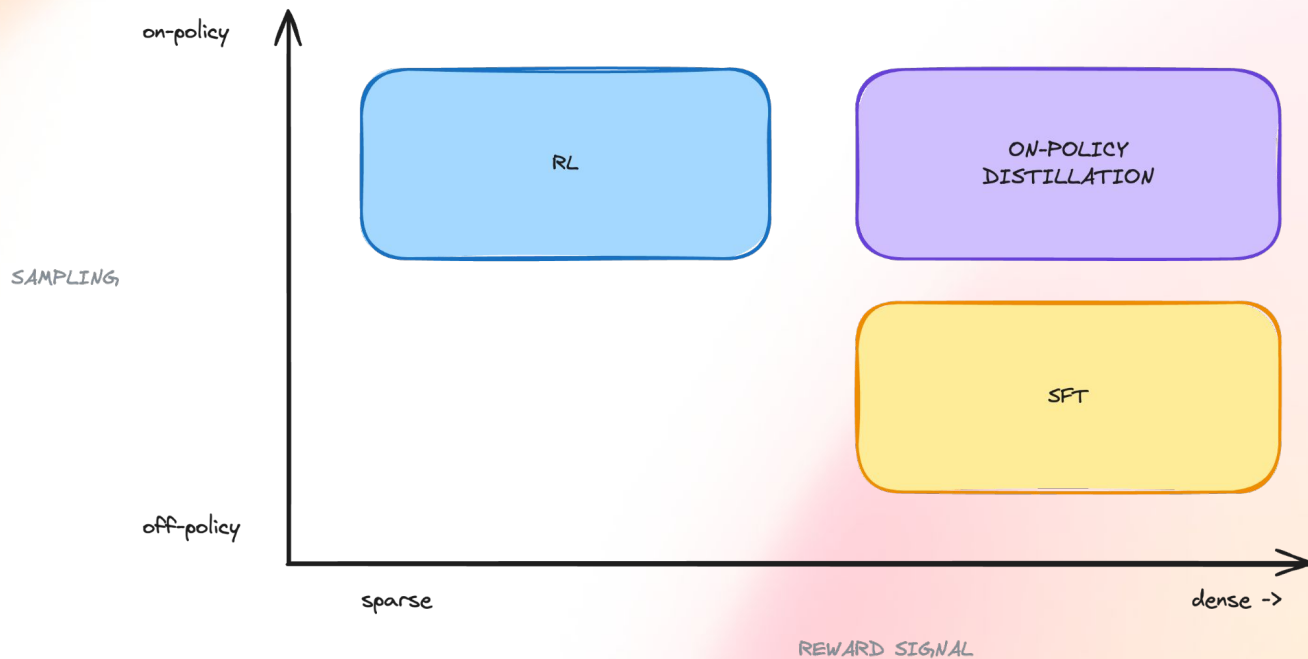


dense feedback on your own play -
including the moves you got wrong

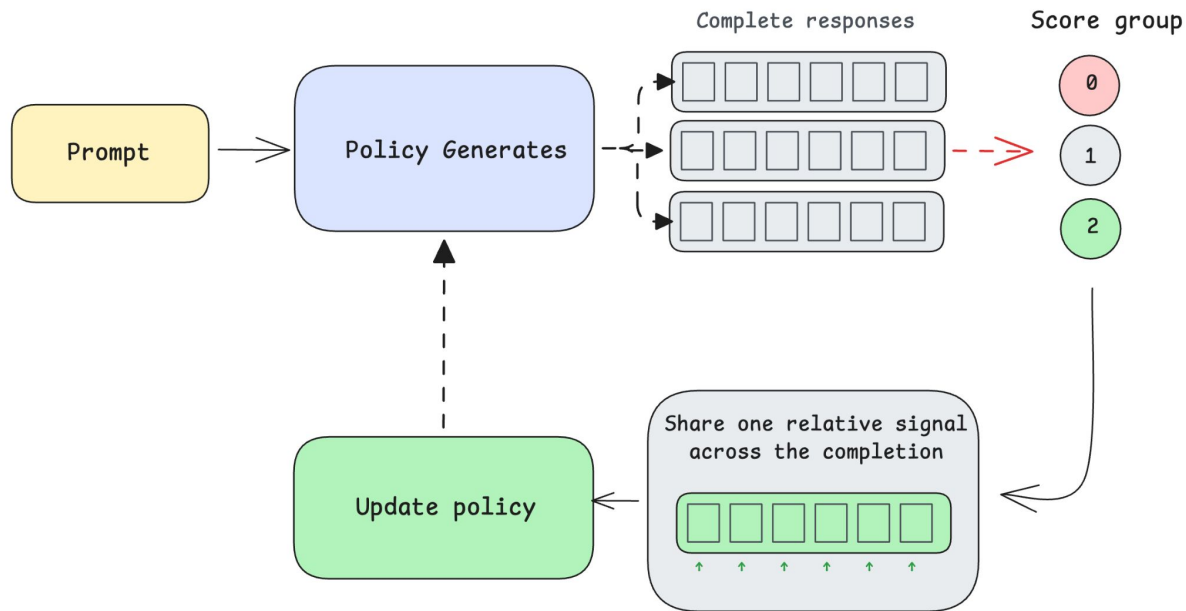
on-policy - dense



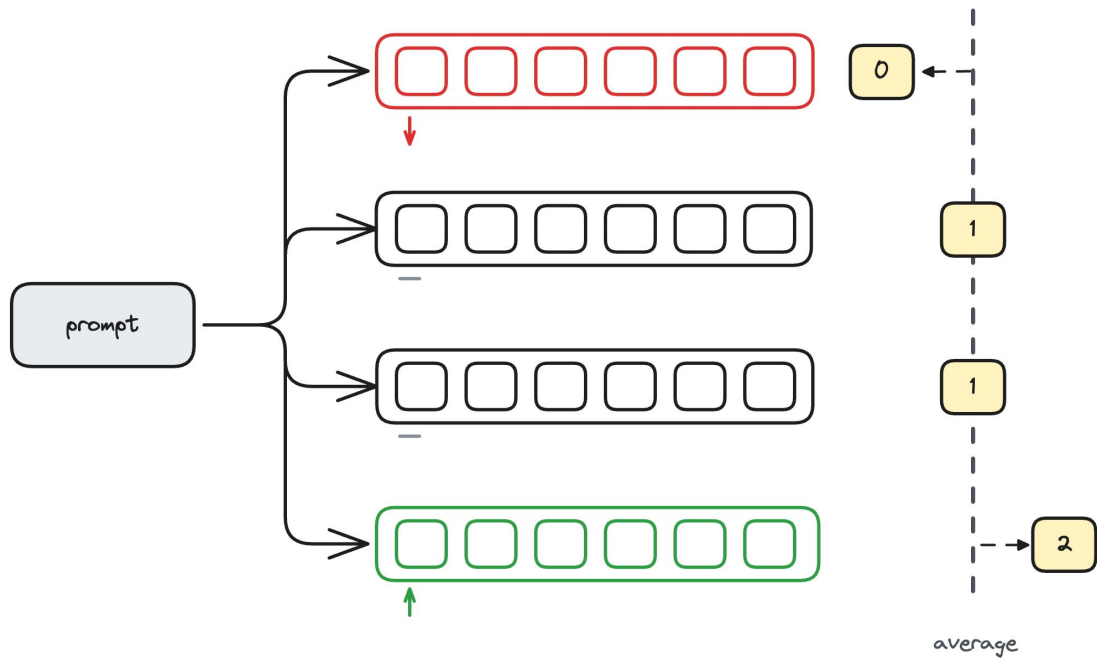
Distillation Landscape



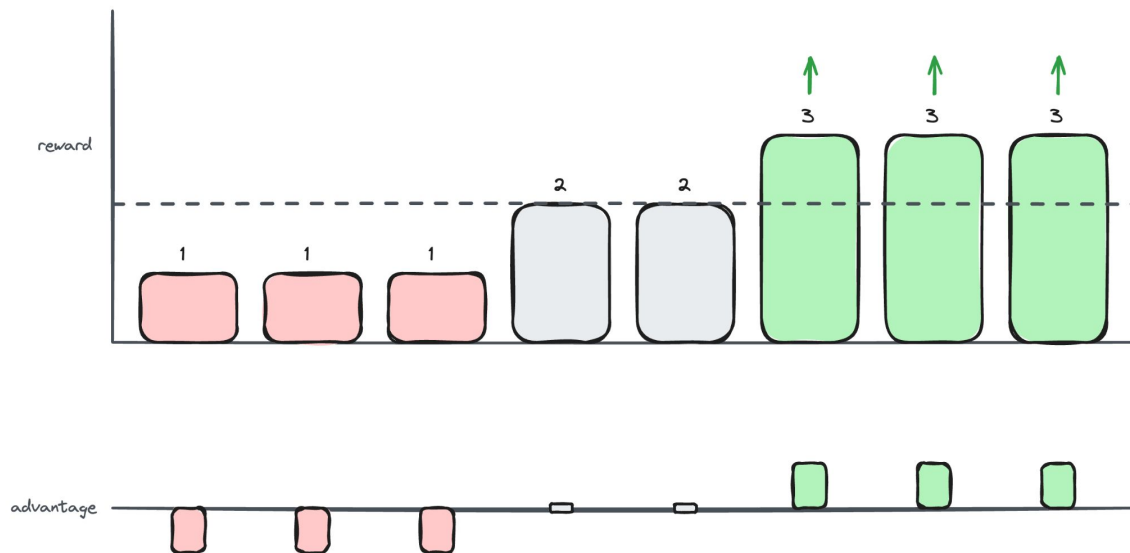
One GRPO training round



The group is the baseline

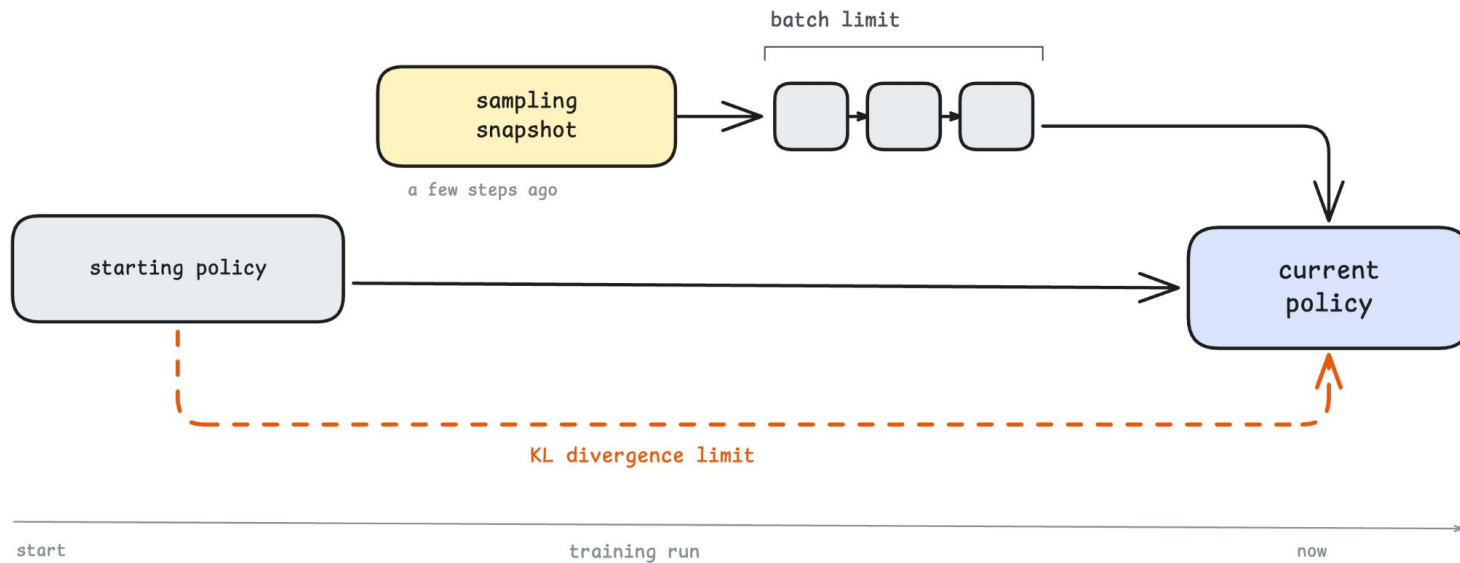


Scores become advantages



$$\text{advantage} = (\text{reward} - \text{group mean}) / (\text{group std})$$

Two guardrails, two time scales



epsilon bounds one reused batch. **beta** bounds the whole run — and its default of 0.0 loads no reference model at all.

The whole update is a few lines

ONE STEP PER ROLLOUT

```
rewards = torch.tensor(
    [reward(r) for r in responses]
)
advantages = (rewards - rewards.mean()) / (
    rewards.std() + 1e-4
)

seq_logp = (token_logp * comp_mask).sum(dim=-1)
loss = -(advantages * seq_logp).mean()
```

Positive advantage raises the completion's log-probability, negative lowers it, and the size sets how hard.

REUSING A ROLLOUT ADDS A CLIP

```
coef_1 = torch.exp(
    per_token_logps - old_per_token_logps
)
coef_2 = torch.clamp(
    coef_1, 1 - epsilon, 1 + epsilon
)
per_token_loss = -torch.min(
    coef_1 * advantages, coef_2 * advantages
)
```

Generation is the slow part, so rollouts get reused. At a single step the ratio is one and the clip does nothing.

The reward is a testable contract

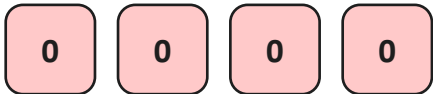
		Reward		
Answer	Reasoning	Format	Accuracy	Total
<code><answer>72</answer></code>	Sound	1.0	2.0	3.0
<code><answer>72</answer></code>	Faulty, a lucky guess	1.0	2.0	3.0
<code><answer>71</answer></code>	Fluent, with one wrong step	1.0	0.0	1.0
"The answer is 72" with no tags	Sound	0.0	0.0	0.0
<code><answer>72</answer></code> followed by more text	Sound	0.0	0.0	0.0

A fluent derivation with a wrong result fails. A lucky guess with the right result passes. The optimizer can only improve what the reward distinguishes.

The reward must create variation

Too hard

nothing is ever solved



advantage = 0

The model cannot reach the reward yet, so every attempt sits on the mean.

Too easy

everything is solved



advantage = 0

The task is already solved, and there is nothing left for the reward to separate.

Usable

the group disagrees

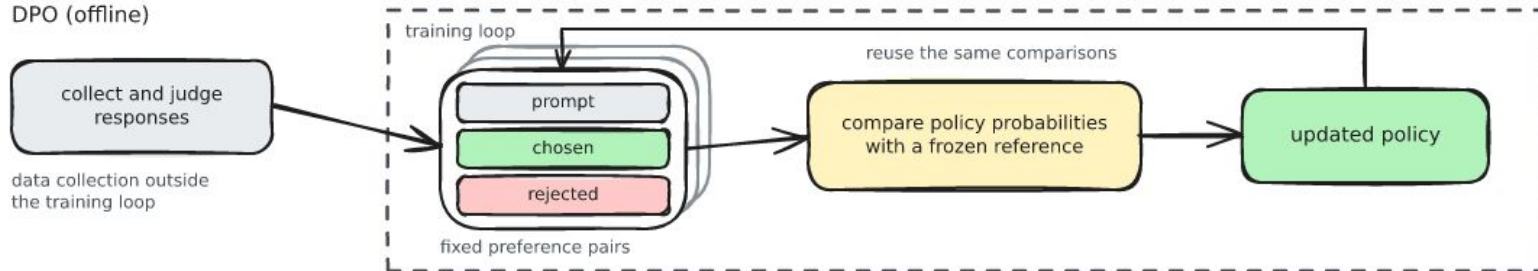


signal

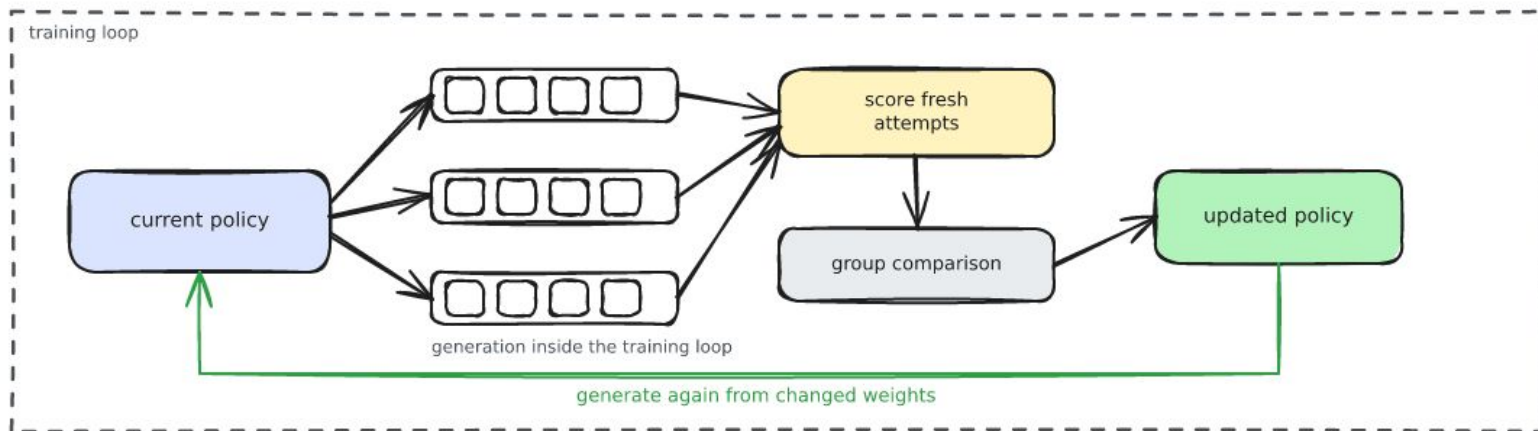
Some attempts beat the mean and some fall short, so the step has a direction to move in.

Fixed preferences or fresh samples

DPO (offline)



GRPO (online)



What GRPO drops: the critic

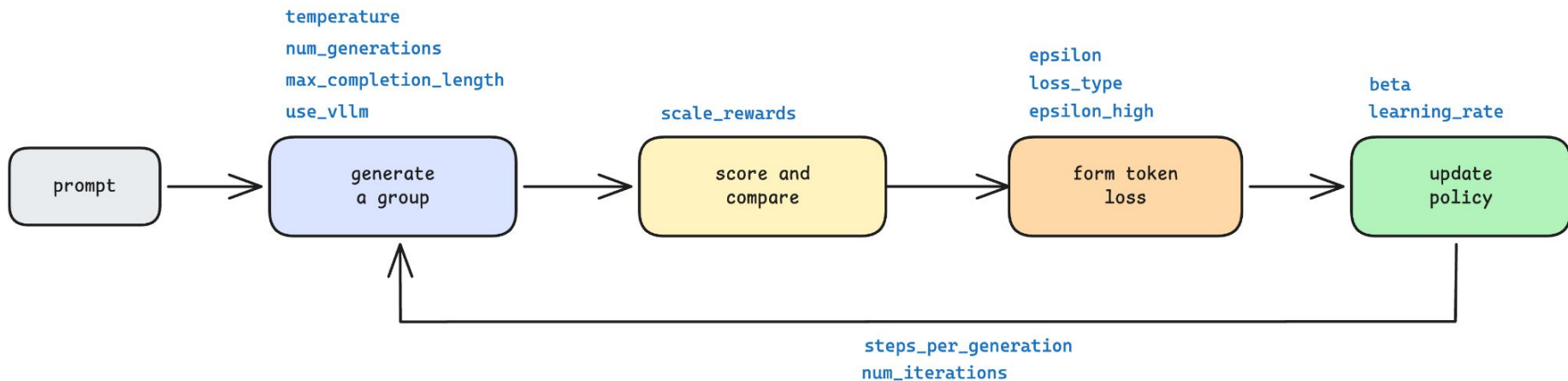
PPO: train a value model

- Predicts the eventual reward after each partial completion, so every token can get its own signal.
- Costs a second model: a language-model backbone, its own optimizer state, and its own training problem.

GRPO: use the sampled group

- The group mean already estimates the expected reward, so there is no second model to hold or fit.
- Costs generation: every prompt needs several completions before there is any signal at all.

The same loop wrapped in **GRPOTrainer** and **GRPOConfig**

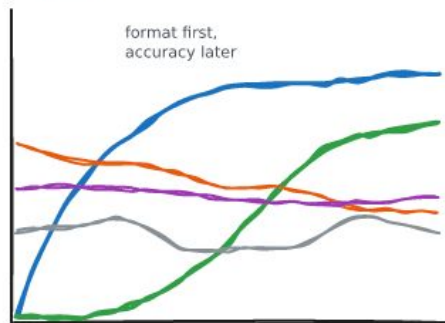


```
trainer = GRPOTrainer(  
    model="Qwen/Qwen3-0.6B",  
    reward_funcs=[format_reward, accuracy_reward],  
    args=GRPOConfig(num_generations= 8, beta=0.0),  
    train_dataset=dataset,  
)
```

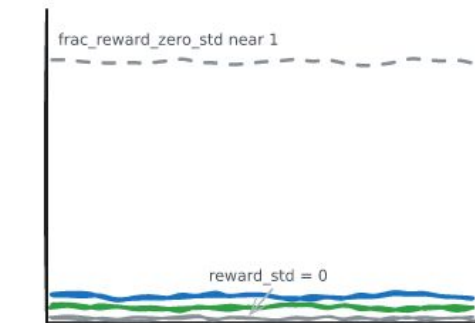
Reading the run

— format reward — accuracy reward — reward spread — entropy — completion length — kl (beta > 0)

healthy



stalled



check task difficulty, reward granularity, group size, and temperature

collapsing



inspect rewarded text first
check epsilon_high and the learning rate
check loss_type and truncation

Experiments



GRPO in *TRL*

```
from trl import GRPOTrainer, GRPOConfig # AsyncGRPOTrainer for harness/multi-turn

def tests_pass_reward(completions, **kwargs):
    # RLVR: verifiable reward, do the tests pass?
    return [1.0 if run_tests(c) else 0.0 for c in completions]

trainer = GRPOTrainer(
    model="Qwen/Qwen3-0.6B",
    reward_funcs=[tests_pass_reward],
    train_dataset=..., # coding tasks
)
trainer.train() # attempts that pass the tests get reinforced
```

Docs: https://huggingface.co/docs/trl/en/grpo_trainer



The smallest GRPO example (exp 1)

Before any real run, let's reward the length of the completion
→ be close to 20 characters long

GRPO optimizes **exactly** the *number* (reward) you return

Quickly, the model (Qwen3-0.6B) optimizes for that goal

```
def reward_len(completions, **kwargs):  
    return [-abs(20 - len(completion)) for completion in completions]
```

Dataset: <https://huggingface.co/datasets/trl-lib/tldr>

Scripts: <https://huggingface.co/sergiopaniego/grpo-youtube-livestream-3-scripts>

Trackio: <https://huggingface.co/spaces/sergiopaniego/trackio-training-agents-3>



GRPO for coding example (exp 2)

MBPP dataset: short Python problems, each with unit tests

The model writes the function. We run the asserts.

reward = fraction that pass

```
Write a function to find the shared elements from the given two lists.
```

```
assert similar_elements((3, 4, 5, 6), (5, 7, 4, 10)) == (4, 5)  
assert similar_elements((1, 2, 3, 4), (5, 4, 3, 7)) == (3, 4)  
assert similar_elements((11, 12, 14, 13), (17, 15, 14, 13)) == (13, 14)
```

This is RLVR: reinforcement learning with verifiable rewards. No reward model, no human labels



GRPO for coding example (exp 2)

```
def tests_reward(completions, test_list, **kwargs):
    rewards = []
    for completion, tests in zip(completions, test_list):
        code = extract_code(completion[0]["content"])
        passed = 0
        for test in tests:
            # "assert similar_elements(...) == ..."
            if runs_ok(code + "\n" + test): # subprocess + timeout
                passed += 1
        rewards.append(passed / len(tests))
    return rewards
```



Does the task give signal? (exp 2)

If every attempt in a group gets the same reward, the advantage is 0 → nothing to learn. You want the task at the edge of the model's ability

- All 0 → task too hard
- Mixed rewards → the signal!
- All 1 → task too easy

Before training Qwen3-0.6B on MBPP, $k=4$ attempts per prompt → mean reward 0.22, and 10 of 20 prompt groups mixed

We trained 3 different versions (num_generations)



Three different versions (exp 2)

The knob is the `num_generations` (group size)

`k=2`, `k=8`, `k=16`

`k=2` → compares each attempt against a single sibling

`k=16` → compares against a crowd

`reward` → the target-task signal. Should climb 

Dataset: <https://huggingface.co/datasets/google-research-datasets/mbpp>

Scripts: <https://huggingface.co/sergiopaniego/grpo-youtube-livestream-3-scripts>

Trackio: <https://huggingface.co/spaces/sergiopaniego/trackio-training-agents-3>



Failed example (exp 3)

The reward can be gamed

Reward hacking → model maximizes the reward ≠ solving task

Reward  (training), real quality  (test)

```
def gameable_reward(completions, **kwargs):
    """Number of python code blocks in the completion, capped at 4. Deliberately
    gameable."""
    rewards = []
    for completion in completions:
        blocks = re.findall(r"```(?:python)?\n.*?```", completion[0]["content"], re.DOTALL)
        rewards.append(min(len(blocks), 4) / 4)
    return rewards
```

Dataset: <https://huggingface.co/datasets/google-research-datasets/mbpp>

Scripts: <https://huggingface.co/sergiopaniego/grpo-youtube-livestream-3-scripts>

Trackio: <https://huggingface.co/spaces/sergiopaniego/trackio-training-agents-3>



Recap

Imitation to outcomes

SFT copies demonstrations. RL trains on results.

Group sampling

Sample a group of completions per prompt. Score each one.

Relative advantage

The group mean is the baseline. No reward model, no critic.

Verifiable rewards

Rewards are plain functions that check the output.

TRL on HF Jobs

GRPOTrainer runs the whole loop on Hugging Face Jobs.

Read the curves

Reward, KL, and completion length reveal hacking, collapse, and length gaming.